

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA1007	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Scenario-Based Requirement Analysis
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills</i>		
Student Name:	Tharankeswaran M	Reg. No.:	192424081
Date:	28/7/26	Duration:	60 minutes

Code of Conduct

I Tharankeswaran M (192424081) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Tharankeswaran M

Annexure A – Scenario-Based Requirement Analysis

Instructions to Students:

Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

- 1. Study and Analyze the Scenario**
 - Carefully understand the given problem statement.
 - Identify the business domain, stakeholders, existing challenges, and system objectives.
- 2. Identify Functional and Non-Functional Requirements**
 - List all functional requirements required by the proposed system.
 - Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.
- 3. Identify Actors and Use Cases**
 - Identify all primary and secondary actors interacting with the system.
 - List the major use cases associated with each actor.
 - Draw a Use Case Diagram representing the interactions between actors and the system.
- 4. Prepare the Software Requirement Specification (SRS)**

Develop an SRS document containing:

 - Introduction
 - Problem Description
 - Scope of the System

- Functional Requirements
- Non-Functional Requirements
- Use Case Descriptions
- Data Requirements
- External Interface Requirements
- System Features

5. Identify Assumptions and Constraints

- List the assumptions considered while developing the system.
- Identify technical, operational, economic, legal, and time-related constraints affecting the project.

6. Validate Requirement Completeness and Consistency

- Verify whether all stakeholder requirements have been addressed.
- Check for ambiguity, redundancy, inconsistency, and missing requirements.
- Suggest improvements to enhance the quality and completeness of the requirement specification.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification	Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.
Actor Identification and Use Case Modeling	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement Specification (SRS) Documentation	SRS is well-structured, complete, professionally organized, and includes all required sections.	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or lacks essential components.

Assumptions, Constraints, and Requirement Validation	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.
---	---	--	---	--

Criteria	Mark
Scenario Understanding and Problem Analysis	/5
Functional and Non-Functional Requirement Identification	/5
Actor Identification and Use Case Modeling	/5
Software Requirement Specification (SRS) Documentation	/5
Assumptions, Constraints, and Requirement Validation	/5
TOTAL	/5

1 INTRODUCTION

1.1 Purpose

This Software Requirement Specification (SRS) document defines the complete requirements for the **Smart Sports Stadium Operations Platform**. It serves as the authoritative baseline for stakeholders, developers, testers, and project reviewers.

1.2 Document Conventions

Requirements are prioritized as High (H), Medium (M), or Low (L). Each requirement carries a unique identifier: FR-XX (Functional) or NFR-XX (Non-Functional).

1.3 Intended Audience

Stadium management, software architects, development team, security personnel, and regulatory auditors.

1.4 Project Overview

The platform digitizes stadium operations through real-time IoT monitoring, predictive analytics, and integrated command-and-control capabilities to enhance safety, efficiency, and revenue.

2 PROBLEM DESCRIPTION

Current stadium management relies on fragmented manual processes and disconnected legacy systems. The specific challenges are:

- **Safety Risks:** Inability to detect crowd anomalies in real time leads to stampedes and medical emergencies.
- **Inefficient Flows:** Entry and exit bottlenecks cause spectator delays exceeding 45 minutes during peak periods.
- **Security Silos:** Security teams lack centralized coordination, delaying incident response times.
- **Parking Chaos:** Manual parking oversight results in underutilization (approx. 30%) and external traffic congestion.
- **Revenue Loss:** Concession stock-outs and long queues reduce per-capita spending by up to 20%.
- **Data Blindness:** Post-event reviews lack quantitative operational insights for continuous improvement.

3 SCOPE OF THE SYSTEM

3.1 In-Scope

- Real-time crowd density monitoring and movement tracking via computer vision and IoT sensors.
- Dynamic entry/exit gate scheduling with digital signage control.
- Unified security operations center with incident logging, dispatch, and escalation.
- Automated parking guidance and zone-based availability updates.
- Concession sales tracking, inventory alerts, and queue-length monitoring.
- Event analytics dashboard with exportable post-event reports.

3.2 Out-of-Scope

- Primary ticketing and payment processing (external integration only).
- Player or team performance analytics.
- Structural stadium maintenance and facilities management.

4 FUNCTIONAL REQUIREMENTS

FR-1 **Crowd Monitoring:** The system shall display real-time crowd density heat-maps with $\geq 95\%$ accuracy, updating every 5 seconds.

FR-2 **Movement Tracking:** The system shall track spectator flow patterns between zones and detect abnormal congregation or contra-flow.

- FR-3 **Entry/Exit Optimization:** The system shall dynamically adjust gate staffing recommendations and digital signage based on predicted queue lengths.
- FR-4 **Security Coordination:** The system shall enable security officers to report incidents, request backup, and receive dispatch orders through a mobile interface.
- FR-5 **Alert Generation:** The system shall automatically generate tiered alerts (Info, Warning, Critical) when density or behavior thresholds are breached.
- FR-6 **Parking Management:** The system shall display real-time parking availability by zone and guide spectators via mobile app and digital signage.
- FR-7 **Concession Tracking:** The system shall record transaction volumes, average service times, and inventory levels per concession stand.
- FR-8 **Performance Dashboard:** The system shall provide role-based dashboards tailored to managers, security, and administrators.
- FR-9 **Event Analytics:** The system shall generate post-event reports including attendance patterns, safety incidents, revenue per capita, and flow efficiency metrics.
- FR-10 **User Management:** The system shall support role-based access control (RBAC) with at least five distinct operational roles.
- FR-11 **External Integration:** The system shall integrate with emergency services APIs for automated incident escalation.
- FR-12 **Historical Data:** The system shall retain operational data for a minimum of 24 months to support trend analysis.

5 NON-FUNCTIONAL REQUIREMENTS

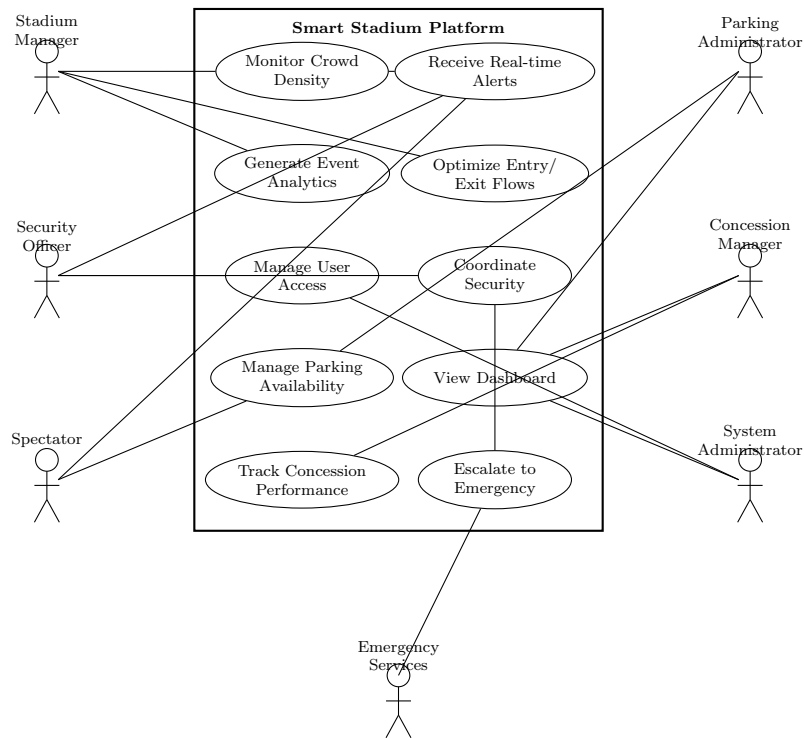
- NFR-1 **Performance:** End-to-end sensor data processing latency shall remain under 2 seconds during events with 50,000 concurrent spectators.
- NFR-2 **Availability:** The system shall maintain 99.9% uptime during scheduled events, with automatic failover to redundant nodes.
- NFR-3 **Scalability:** The architecture shall linearly support stadium capacities from 10,000 to 100,000 spectators without degradation.
- NFR-4 **Reliability:** Mean Time Between Failures (MTBF) shall exceed 2,000 hours; Mean Time To Repair (MTTR) shall be under 15 minutes.
- NFR-5 **Security:** All data in transit shall use TLS 1.3; data at rest shall be encrypted with AES-256; administrative access shall enforce MFA.
- NFR-6 **Usability:** Security officers shall complete an incident report with no more than 3 taps/clicks; dashboard initial load time shall be under 3 seconds.
- NFR-7 **Maintainability:** The system shall adopt a microservices architecture with containerized deployment; unit test coverage shall exceed 80%.
- NFR-8 **Interoperability:** The system shall expose RESTful APIs and support MQTT/CoAP for IoT device communication.
- NFR-9 **Compliance:** The system shall comply with GDPR, local data protection statutes, and stadium safety regulations.
- NFR-10 **Portability:** The mobile interface shall support Android 10+ and iOS 14+ devices.

6 ACTORS AND USE CASES

6.1 Actor Identification

Actor	Type	Description
Stadium Manager	Primary	Oversees overall operations, reviews analytics, and makes strategic decisions.
Security Officer	Primary	Patrols stadium, reports incidents, and responds to real-time alerts.
Parking Administrator	Primary	Manages parking zones, monitors occupancy, and adjusts pricing.
Concession Manager	Primary	Tracks sales, inventory, and queue performance per stand.
Spectator	Primary	Views parking availability and receives safety notifications via mobile app.
System Administrator	Secondary	Manages accounts, roles, system configuration, and backups.
Emergency Services	Secondary	Receives automated critical incident alerts and dispatches external response.

6.2 Use Case Diagram



6.3 Use Case Descriptions

UC-1: Monitor Crowd Density

- **Actors:** Stadium Manager, Security Officer
- **Precondition:** IoT sensors and cameras are calibrated; event is active.
- **Flow:** System aggregates sensor feeds → computes density heat-map → renders on dashboard → triggers alert if threshold breached.
- **Postcondition:** Operations team has real-time crowd visibility.

UC-2: Optimize Entry/Exit Flows

- **Actor:** Stadium Manager
- **Precondition:** Gate sensors operational; event schedule loaded.
- **Flow:** System predicts queue lengths using historical and real-time data → recommends gate staffing and signage updates → manager approves or overrides.
- **Postcondition:** Spectator ingress/egress wait times are minimized.

UC-3: Coordinate Security Operations

- **Actors:** Security Officer, Emergency Services
- **Precondition:** Security personnel authenticated on mobile app.
- **Flow:** Officer reports incident → system logs and geotags → dispatches nearest units → auto-escalates to emergency services if severity = Critical.
- **Postcondition:** Incident is contained, documented, and closed.

UC-4: Manage Parking Availability

- **Actors:** Parking Administrator, Spectator
- **Precondition:** Parking zone sensors active and calibrated.
- **Flow:** System detects vehicle entry/exit → updates zone counts → pushes availability to mobile app and digital signage.
- **Postcondition:** Spectators are directed to available spaces efficiently.

UC-5: Track Concession Performance

- **Actor:** Concession Manager
- **Precondition:** POS systems integrated and streaming data.
- **Flow:** System collects sales and queue data → computes KPIs (revenue, throughput, wait time) → alerts on stock-outs → generates shift report.
- **Postcondition:** Manager has actionable concession insights.

UC-6: Generate Event Analytics

- **Actor:** Stadium Manager
- **Precondition:** Event concluded; data validation complete.
- **Flow:** System aggregates operational data → computes safety, flow, and revenue metrics → exports PDF/Excel report with visualizations.
- **Postcondition:** Post-event review document is available for stakeholders.

7 DATA REQUIREMENTS

7.1 Data Entities

- **Spectator Data:** Anonymized movement vectors, zone dwell times, entry/exit timestamps.
- **Sensor Data:** Camera metadata, occupancy counts, noise/temperature readings.
- **Parking Data:** Zone ID, total capacity, current occupancy, reservation flags.
- **Concession Data:** Stand ID, SKU inventory, transaction logs, queue length estimates.
- **Security Data:** Incident type, severity, GPS location, timestamp, responding officer, resolution status, media attachments.
- **Event Data:** Event ID, type, scheduled/actual times, attendance count, weather correlation.
- **User Data:** Role, credentials (hashed), permissions, audit logs, session tokens.

7.2 Data Storage and Retention

- **Real-time cache:** Redis, 24-hour retention for sub-second dashboard queries.
- **Operational store:** PostgreSQL, 24-month retention with monthly partitioning.
- **Video archives:** Object storage (S3-compatible), 90-day hot retention then tiered to cold storage.

8 EXTERNAL INTERFACE REQUIREMENTS

8.1 User Interfaces

- **Web Dashboard:** Responsive HTML5/CSS3 interface for managers and administrators (Chrome, Firefox, Edge).
- **Mobile App:** Native iOS/Android apps for security officers and spectators.
- **Digital Signage:** REST endpoint pushing JSON payloads to stadium LED/display controllers.

8.2 Hardware Interfaces

- **IoT Sensors:** MQTT 5.0 for occupancy sensors; RTSP/RTP for IP camera streams.
- **POS Terminals:** Secure HTTPS API integration with concession vendor systems.
- **Parking Gates:** Relay control interfaces and inductive loop vehicle detectors.

8.3 Software Interfaces

- **Ticketing System:** REST API to ingest expected attendance and gate assignments.
- **Emergency Services:** SOAP/REST gateway for automated critical incident reporting.
- **Weather Service:** Public REST API for real-time weather correlation with crowd behavior.

8.4 Communications Interfaces

- HTTPS/TLS 1.3 for all web and API traffic.
- WebSocket connections for real-time dashboard updates.
- SMS/Push notification gateway for critical safety alerts.

9 SYSTEM FEATURES

1. **Real-Time Command Center:** Unified geospatial dashboard displaying crowd, parking, security, and concession status with zone-level drill-down.
2. **Predictive Flow Engine:** ML-based module forecasting congestion 15–30 minutes in advance using historical patterns and real-time feeds.
3. **Incident Management Module:** End-to-end digital workflow for reporting, dispatching, tracking, and closing security incidents with photo/video evidence.
4. **Smart Parking Guidance:** Dynamic wayfinding integrating mobile apps, variable message signs, and LED bay indicators to minimize search time.
5. **Concession Intelligence:** Automated low-stock alerts, dynamic pricing suggestions, and demand-forecasting per stand.
6. **Post-Event Analytics Suite:** Comparative analysis across events, exportable visualizations, and automated executive summary generation.

10 ASSUMPTIONS AND CONSTRAINTS

10.1 Assumptions

- High-bandwidth network infrastructure (Wi-Fi 6 / 5G) covers all stadium zones including parking.
- Stadium management commits to procuring and installing IoT sensors and IP cameras.
- Spectator smartphone penetration exceeds 80% for effective mobile app adoption.
- Concession vendors agree to integrate their POS systems via standardized APIs.
- Regulatory approval for video analytics in public spaces is obtainable without biometric identification.

10.2 Constraints

- **Technical:** The system must ingest and process >10,000 sensor events per second during peak load without data loss.
- **Economic:** Development and first-year deployment are bound by a fixed capital budget; open-source components are preferred.
- **Legal:** Full compliance with GDPR and local data privacy laws is mandatory; facial recognition is explicitly excluded.
- **Operational:** The platform must not interfere with existing safety certifications, broadcast systems, or PA infrastructure.
- **Time:** Initial production deployment must be completed within 6 months, before the next major sporting season.

11 REQUIREMENT VALIDATION

11.1 Completeness Check

All six stated objectives—crowd monitoring, flow optimization, security coordination, parking management, concession tracking, and event analytics—are explicitly mapped to functional requirements FR-1 through FR-12. Every stakeholder group identified in the actor table is represented by at least one primary use case.

11.2 Consistency Verification

- Terminology is standardized: “crowd density” consistently refers to persons per 10 m².
- No contradictory requirements: FR-5 (automatic alerts) and NFR-1 (2-second latency) are architecturally aligned.
- Units are uniform throughout: time in seconds, capacity in spectators, retention in months.

11.3 Ambiguity Review

- Performance targets are fully quantified (95% accuracy, 2-second latency, 99.9% uptime, 2,000-hour MTBF).
- Alert thresholds are defined as configurable per event type rather than fixed constants.
- RBAC roles are explicitly enumerated, eliminating access-control ambiguity.

11.4 Suggested Improvements

- Add explicit accessibility requirements (WCAG 2.1 AA compliance) for the web dashboard and mobile apps.
- Define disaster recovery objectives (RPO/RTO) as separate non-functional requirements.
- Include usability testing milestones with actual security officers and spectators prior to final acceptance.
- Specify API rate limits, versioning strategy, and deprecation policies for all external integrations.